```
#Amritansh Singh 23070521013
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Input
from tensorflow.keras import regularizers

# --- 1. Use the User's Specified Dataset ---
# Original data provided by the user
x_user = np.array([1, 2, 3, 4, 5])
y_user = np.array([20, 40, 50, 65, 80])

# Reshape x for model input (sklearn expects 2D array for features)
X = x_user.reshape(-1, 1)
y = y_user

print("User's dataset loaded.")
print(f"X shape: {X.shape}, y shape: {y.shape}")

# --- 2. Preprocessing ---
# With such a small dataset, splitting into train/test might not be representative.
# For demonstration, we'll use the entire dataset for training and testing.
# In a real scenario, you'd need more data or cross-validation.

# We will scale the single feature for consistency with deep learning practices
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# For simplicity, and due to the tiny dataset size, we'll train and evaluate on the full dataset.
# In a real application, proper train/test/validation splits are crucial.
X_train_scaled, y_train = X_scaled, y
X_test_scaled, y_test = X_scaled, y # Using same data for test for demonstration with tiny set

print("Data scaled (entire dataset used for train/test).")

# --- 3. Build and Train a Baseline Model (No Regularization) ---
def build_baseline_model(input_shape):
    model = Sequential([
        Input(shape=(input_shape,)), # Explicit Input layer
        Dense(64, activation='relu'),
        Dense(32, activation='relu'),
        Dense(1) # Output layer for regression
    ])
    model.compile(optimizer='adam', loss='mse', metrics=['mae'])
    return model

print("\n--- Training Baseline Model ---")
baseline_model = build_baseline_model(X_train_scaled.shape[1])
history_baseline = baseline_model.fit(
    X_train_scaled, y_train, epochs=200, batch_size=len(X_train_scaled), verbose=0 # Use higher epochs, full batch
)

# Evaluate Baseline Model
loss_baseline, mae_baseline = baseline_model.evaluate(X_test_scaled, y_test, verbose=0)
print(f"Baseline Model - Test MAE: {mae_baseline:.2f}")

# --- 4. Build and Train Model with L2 Regularization ---
def build_l2_model(input_shape, l2_lambda=0.01):
    model = Sequential([
        Input(shape=(input_shape,)), # Explicit Input layer
        Dense(64, activation='relu', kernel_regularizer=regularizers.l2(l2_lambda)),
        Dense(32, activation='relu', kernel_regularizer=regularizers.l2(l2_lambda)),
        Dense(1)
    ])
    model.compile(optimizer='adam', loss='mse', metrics=['mae'])
    return model

print("\n--- Training L2 Regularized Model ---")
l2_model = build_l2_model(X_train_scaled.shape[1])
history_l2 = l2_model.fit(
    X_train_scaled, y_train, epochs=200, batch_size=len(X_train_scaled), verbose=0
)

# Evaluate L2 Model
loss_l2, mae_l2 = l2_model.evaluate(X_test_scaled, y_test, verbose=0)
print(f"L2 Regularized Model - Test MAE: {mae_l2:.2f}")
```

```
# --- 5. Build and Train Model with Dropout ---
def build_dropout_model(input_shape, dropout_rate=0.2):
    model = Sequential([
        Input(shape=(input_shape,)), # Explicit Input layer
        Dense(64, activation='relu'),
        Dropout(dropout_rate),
        Dense(32, activation='relu'),
        Dropout(dropout_rate),
        Dense(1)
    ])
    model.compile(optimizer='adam', loss='mse', metrics=['mae'])
    return model

print("\n--- Training Dropout Model ---")
dropout_model = build_dropout_model(X_train_scaled.shape[1])
history_dropout = dropout_model.fit(
    X_train_scaled, y_train, epochs=200, batch_size=len(X_train_scaled), verbose=0
)

# Evaluate Dropout Model
loss_dropout, mae_dropout = dropout_model.evaluate(X_test_scaled, y_test, verbose=0)
print(f"Dropout Model - Test MAE: {mae_dropout:.2f}")

# --- 6. Build and Train Model with L1 Regularization ---
def build_l1_model(input_shape, l1_lambda=0.01):
    model = Sequential([
        Input(shape=(input_shape,)), # Explicit Input layer
        Dense(64, activation='relu', kernel_regularizer=regularizers.l1(l1_lambda)),
        Dense(32, activation='relu', kernel_regularizer=regularizers.l1(l1_lambda)),
        Dense(1)
    ])
    model.compile(optimizer='adam', loss='mse', metrics=['mae'])
    return model

print("\n--- Training L1 Regularized Model ---")
l1_model = build_l1_model(X_train_scaled.shape[1])
history_l1 = l1_model.fit(
    X_train_scaled, y_train, epochs=200, batch_size=len(X_train_scaled), verbose=0
)

# Evaluate L1 Model
loss_l1, mae_l1 = l1_model.evaluate(X_test_scaled, y_test, verbose=0)
print(f"L1 Regularized Model - Test MAE: {mae_l1:.2f}")

# --- 7. Compare Performance (Updated to include L1) ---
print("\n--- Performance Comparison (Mean Absolute Error) ---")
print(f"Baseline Model MAE: {mae_baseline:.2f}")
print(f"L2 Regularized Model MAE: {mae_l2:.2f}")
print(f"Dropout Model MAE: {mae_dropout:.2f}")
print(f"L1 Regularized Model MAE: {mae_l1:.2f}")

# For such a small dataset, these models are likely to overfit significantly.
# The MAE values should be interpreted with caution due to the limited data.
```

```
User's dataset loaded.
X shape: (5, 1), y shape: (5,)
Data scaled (entire dataset used for train/test).

--- Training Baseline Model ---
Baseline Model - Test MAE: 19.20

--- Training L2 Regularized Model ---
L2 Regularized Model - Test MAE: 20.21

--- Training Dropout Model ---
Dropout Model - Test MAE: 24.16

--- Training L1 Regularized Model ---
L1 Regularized Model - Test MAE: 23.45

--- Performance Comparison (Mean Absolute Error) ---
Baseline Model MAE: 19.20
L2 Regularized Model MAE: 20.21
Dropout Model MAE: 24.16
L1 Regularized Model MAE: 23.45
```